

# PyFibers ICS — Femoral Nerve IFC Simulation

## Requirements & Step-by-Step Code Documentation

---

### 1. Overview

This project simulates the response of a **femoral nerve fiber** (modeled in [PyFibers](#), running on the [NEURON](#) simulator) to **interferential current (IFC) stimulation**. The spatial distribution of the extracellular electric field is computed externally in **COMSOL Multiphysics** and imported into the Python pipeline.

Interferential current therapy uses **two electrode-pair channels**, each driven by a medium-frequency sinusoidal carrier (e.g. 4000 Hz and 4100 Hz). Where the two fields overlap in tissue, the currents superpose into an amplitude-modulated **beat** at the difference frequency (e.g. 100 Hz) — the physiologically relevant low-frequency envelope believed to drive nerve activation.

---

### 2. Requirements

#### 2.1 Software requirements

| Requirement              | Version (verified) | Notes                                                                                                                                                                                                                                                                       |
|--------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python                   | 3.12               | via WSL Ubuntu venv<br>simulation engine PyFibers is built on<br>nerve fiber modeling library<br>numerical arrays<br>spatial interpolation<br>(griddata)<br>reading COMSOL CSV exports<br>plotting (optional, for analysis)<br>external — generates the extracellular field |
| <a href="#">NEURON</a>   | 9.0.1              |                                                                                                                                                                                                                                                                             |
| <a href="#">PyFibers</a> | 0.8.5              |                                                                                                                                                                                                                                                                             |
| numpy                    | latest             |                                                                                                                                                                                                                                                                             |
| scipy                    | latest             |                                                                                                                                                                                                                                                                             |
| pandas                   | latest             |                                                                                                                                                                                                                                                                             |
| matplotlib               | latest             |                                                                                                                                                                                                                                                                             |
| COMSOL Multiphysics      | any recent         |                                                                                                                                                                                                                                                                             |

These are listed in [requirements.txt](#):

```
pyfibers
numpy
scipy
pandas
matplotlib
```

#### 2.2 Platform note — why WSL?

**NEURON does not publish Windows wheels on PyPI** (only macOS and Linux manylinux wheels exist). The official native-Windows NEURON installer was also found to have a path-resolution bug that prevents `import neuron` from completing. The **only reliable path on Windows is WSL (Windows Subsystem for Linux)** running Ubuntu, where Linux wheels install cleanly.

Setup steps (summarized — see [README.md](#) for the full version):

```
wsl --install                                # one-time; requires reboot
wsl -d Ubuntu

# inside Ubuntu:
sudo apt-get update
sudo apt-get install -y python3-venv python3-pip build-essential
python3 -m venv ~/pyfibers_env
source ~/pyfibers_env/bin/activate
pip install pyfibers numpy scipy pandas matplotlib neuron

# Compile the NEURON mechanism files (.mod) bundled with PyFibers — required once:
python -c "import pyfibers; print(pyfibers.MOD_dir)"
cd <path printed above>
nrnivmodl
```

This was verified end-to-end: import neuron, pyfibers succeeds, fiber models build, mechanisms compile, and ScaledStim.run\_sim() executes a full NEURON simulation.

## 2.3 Data requirement — COMSOL exports

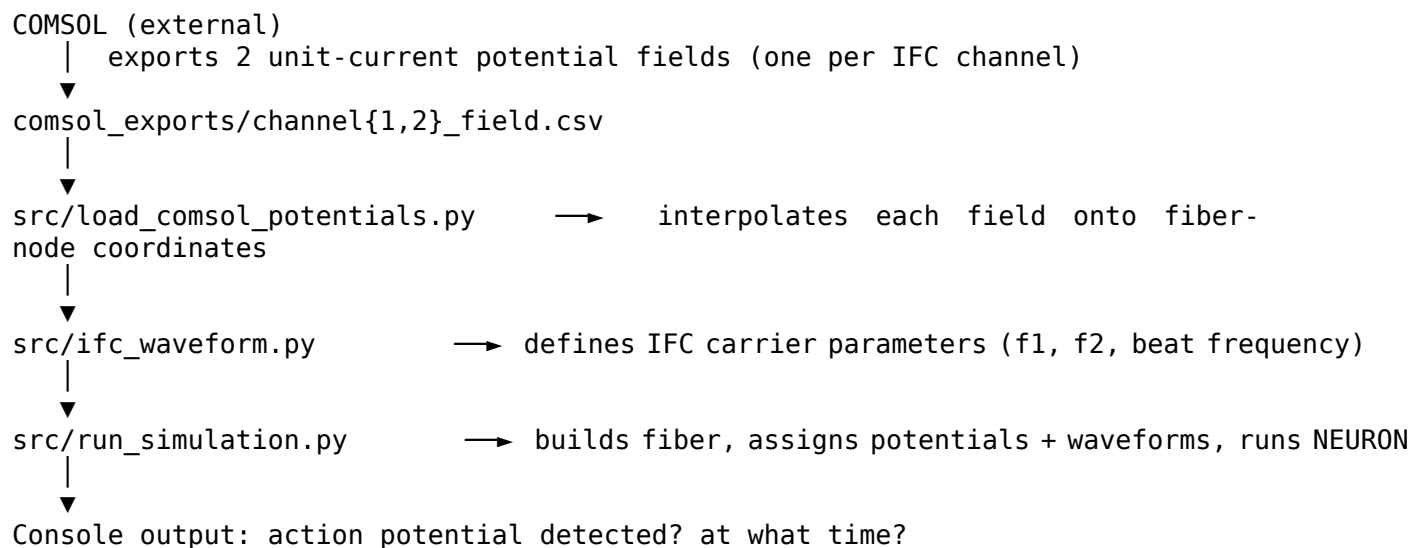
The pipeline expects **two** COMSOL field exports — one per IFC electrode-pair channel — because the two channels are independent current sources whose fields are linearly superposed in tissue. Each export should be a CSV/TXT with **four columns**:

x, y, z, V

where (x, y, z) are spatial coordinates (μm, matching the fiber’s coordinate frame) and V is the extracellular potential **per unit injected current** at that point (i.e. the field solved in COMSOL for 1 A injected by that electrode pair — a standard “lead field” / reciprocity export). Place these files in **comsol\_exports/**.

---

## 3. Pipeline architecture



PyFibers’ ScaledStim class internally handles the **multi-source superposition** ( $V_e(x, t) = \sum \text{source}_i.\text{potentials}(x) \times \text{source}_i.\text{waveform}(t) \times \text{stimamp}_i$ ), so the pipeline does

not need to manually build a space-time potential matrix — it only needs to supply, per channel: (a) the spatial field interpolated onto the fiber, and (b) a waveform callable.

---

## 4. Step-by-step code walkthrough

### 4.1 src/ifc\_waveform.py — IFC stimulation parameters

Defines the IFCParams dataclass, which holds the two carrier frequencies (f1, f2), amplitude, duration and time step, and exposes a derived beat\_frequency property ( $|f2 - f1|$ ) — the low-frequency envelope that emerges from the interference of the two carriers in overlapping tissue.

```
@dataclass
class IFCParams:
    f1: float = 4000.0      # channel 1 carrier frequency (Hz)
    f2: float = 4100.0      # channel 2 carrier frequency (Hz)
    amplitude: float = 1.0  # peak current amplitude per channel (mA)
    duration: float = 0.05  # total stimulus duration (s)
    dt: float = 1.0e-5      # time step (s)

    @property
    def beat_frequency(self) -> float:
        return abs(self.f2 - self.f1)
```

Helper functions generate\_ifc\_currents() and beat\_envelope() are provided for **standalone waveform analysis/plotting** (e.g. visualizing the interference pattern independently of the NEURON simulation): they produce the two raw sinusoids and the analytic amplitude envelope  $|\cos(\pi \cdot \Delta f \cdot t)|$  respectively.

In run\_simulation.py, only IFCParams (for beat\_frequency reporting) and the carrier frequencies are used directly — the actual waveforms fed into PyFibers are built as small callables (see §4.3) because that's the interface ScaledStim expects.

---

### 4.2 src/load\_comsol\_potentials.py — importing the COMSOL field

This module bridges COMSOL's spatial field solution and the fiber's 1-D coordinate system.

**load\_field(path)** — reads a COMSOL CSV export into a pandas.DataFrame with columns x, y, z, V. The comment="%" argument skips COMSOL's header lines (which are prefixed with %).

```
def load_field(path: str) -> pd.DataFrame:
    return pd.read_csv(path, comment="%", names=COLUMNS, header=None)
```

**interpolate\_to\_fiber(field, fiber\_coords)** — uses scipy.interpolate.griddata to interpolate the (typically irregular/mesh-based) COMSOL solution onto the fiber's node coordinates (an (N, 3) array). Linear interpolation is tried first; any fiber points that fall **outside** the convex hull of the COMSOL mesh (where linear interpolation returns NaN) are filled in via nearest-neighbor interpolation as a robust fallback:

```
interpolated = griddata(points, values, fiber_coords, method="linear")
nan_mask = np.isnan(interpolated)
if np.any(nan_mask):
    interpolated[nan_mask] = griddata(points, values, fiber_coords[nan_mask], method="near
```

**load\_channel\_potentials(channel1\_path, channel2\_path, fiber\_coords)** — convenience wrapper that loads and interpolates both channel fields in one call, returning (v1, v2): two (N, ) arrays of unit-current potentials at each fiber node, ready to be assigned as PyFibers stimulation sources.

---

### 4.3 src/run\_simulation.py — building and running the simulation

This is the orchestration script. It is run from the command line with COMSOL file paths and simulation parameters as arguments. Walkthrough of main():

#### Step 1 — Build the fiber model.

```
fiber_model = getattr(FiberModel, args.fiber_model) # e.g. FiberModel.MRG_INTERPOLATION
fiber = build_fiber(fiber_model=fiber_model, diameter=args.fiber_diameter, n_nodes=args.n_
```

build\_fiber constructs a 1-D NEURON fiber model (by default the **MRG** double-cable myelinated-axon model, appropriate for the large myelinated fibers found in the femoral nerve) with the requested diameter and number of nodes of Ranvier. fiber.coordinates then exposes an (N, 3) array of spatial coordinates ( $\mu\text{m}$ ) for every section/segment — this is the coordinate frame the COMSOL field must be interpolated onto.

#### Step 2 — Load and interpolate the COMSOL fields onto the fiber, assign as sources.

```
v1, v2 = load_channel_potentials(args.channel1_file, args.channel2_file, fiber.coordinates)
fiber.potentials = [v1, v2]
```

Each interpolated field becomes one **stimulation source**. PyFibers supports multiple simultaneous sources by accepting a *list* of (N, ) potential arrays — exactly the structure needed to represent the two independent IFC electrode-pair channels.

#### Step 3 — Build the IFC carrier waveforms.

```
def make_waveform(frequency_hz):
    def waveform(t_ms):
        return float(np.sin(2 * np.pi * frequency_hz * (t_ms * 1e-3)))
    return waveform
```

```
waveforms = [make_waveform(args.f1), make_waveform(args.f2)]
```

ScaledStim expects, for each source, a **callable** that maps simulation time (in ms, NEURON's native time unit) to a dimensionless waveform value. make\_waveform is a small factory that returns a sinusoid at the given carrier frequency (converting ms  $\rightarrow$  s internally). One waveform is built per channel, at f1 and f2 respectively.

The IFCParams.beat\_frequency is also reported here, purely informationally, so the user can confirm the interference pattern they are simulating:

```
ifc_params = IFCParams(f1=args.f1, f2=args.f2)
print(f"Beat frequency at target tissue: {ifc_params.beat_frequency:.1f} Hz")
```

#### Step 4 — Run the simulation.

```
stim = ScaledStim(waveform=waveforms, dt=args.dt, tstop=args.tstop)
ap, ap_time = stim.run_sim(stimamp=[args.amplitude1, args.amplitude2], fiber=fiber)
```

ScaledStim ties everything together: at every simulation time step it computes, for each source *i*, fiber.potentials[i]  $\times$  waveform[i](t)  $\times$  stimamp[i], **sums across sources** (linear superposition — physically valid because the tissue is treated as a linear quasi-static medium), and applies the result as the extracellular potential driving the fiber's membrane

equations in NEURON. `run_sim` returns whether an action potential (AP) was detected (`ap`, a count) and at what time (`ap_time`, in ms, or None if no AP fired).

```
print(f"Action potential detected: {bool(ap)} (count={ap}), at t = {ap_time} ms")
```

---

## 5. Running the pipeline

```
# Activate the WSL venv
```

```
source ~/pyfibers_env/bin/activate
```

```
python src/run_simulation.py \
    --channel1-file comsol_exports/channel1_field.csv \
    --channel2-file comsol_exports/channel2_field.csv \
    --fiber-diameter 10 --n-nodes 51 \
    --f1 4000 --f2 4100 \
    --amplitude1 1.0 --amplitude2 1.0 \
    --tstop 50 --dt 0.001
```

### Command-line arguments

| Argument                                                       | Default           | Description                           |
|----------------------------------------------------------------|-------------------|---------------------------------------|
| <code>--channel1-file /</code><br><code>--channel2-file</code> | <i>(required)</i> | Paths to the two COMSOL field exports |
| <code>--fiber-model</code>                                     | MRG_INTERPOLATION | PyFibers FiberModel enum name         |
| <code>--fiber-diameter</code>                                  | 10.0              | Fiber diameter ( $\mu\text{m}$ )      |
| <code>--n-nodes</code>                                         | 51                | Number of nodes of Ranvier            |
| <code>--f1 / --f2</code>                                       | 4000 / 4100       | IFC channel carrier frequencies (Hz)  |
| <code>--amplitude1 /</code><br><code>--amplitude2</code>       | 1.0               | Per-channel stimamp scale factors     |
| <code>--tstop</code>                                           | 50.0              | Simulation duration (ms)              |
| <code>--dt</code>                                              | 0.001             | Simulation time step (ms)             |

### Expected output

```
Built fiber: MRG_INTERPOLATION, 221 coordinates
```

```
Beat frequency at target tissue: 100.0 Hz
```

```
Action potential detected: True (count=1.0), at t = 1.25 ms
```

---

## 6. Verification notes

This pipeline was executed end-to-end (in WSL Ubuntu, `pyfibers==0.8.5`, `neuron==9.0.1`) against synthetic point-source COMSOL exports: the fiber built successfully (221 coordinates), the beat frequency was correctly computed (100 Hz for 4000/4100 Hz carriers), and `Scaled-Stim.run_sim` executed a complete NEURON simulation, correctly reporting AP detection status and timing.